

Repository Pattern and Service Pattern Used in the Fleet Vehicle Maintenance API

Repository Pattern

In this project, the Repository Pattern is used to manage all database-related operations. The repository layer acts as a bridge between the application and the database, ensuring that data access logic is separated from business logic. Instead of directly accessing Entity Framework Core and the DbContext from controllers, all database operations are performed through repository classes.

For example, VehicleRepository, DriverRepository, and MaintenanceRepository are responsible for retrieving, adding, and managing data from their respective tables. This approach improves code organization, promotes reusability, and makes the application easier to maintain. It also helps in implementing loose coupling, allowing changes to the data access layer without affecting other parts of the application.

Service Pattern

The Service Pattern is used to implement the business logic of the application. The service layer acts as an intermediary between controllers and repositories. Controllers receive requests from users and pass them to the service layer, where validations, business rules, filtering, sorting, and pagination are handled.

In this project, VehicleService, DriverService, and MaintenanceService are responsible for processing requests and applying business validations before interacting with the repository layer. For example, the MaintenanceService validates page size, page number, sorting fields, sorting direction, and filter parameters before retrieving maintenance records.

Using the Service Pattern keeps controllers lightweight and focused only on handling HTTP requests and responses. It also improves maintainability, readability, and testability of the application.

Conclusion

The combination of Repository Pattern and Service Pattern helps achieve a clean layered architecture. The Repository Layer manages database operations, while the Service Layer handles business logic and validations. This separation of responsibilities improves code quality, maintainability, scalability, and follows industry-standard software development practices.